Faster Sorting Algorithm Part 2

by Malte Skarupke

This is a follow up to my previous blog post about writing a faster sorting algorithm. I'm using this as a chance to go into detail on topics that I was asked about in the comments: I'll clear up some misunderstandings and go into future work that needs to happen with this algorithm.

Somebody was nice enough to link my blog post on Hacker News and Reddit. While I didn't do that, I still read most of the comments on those website. For some reasons the comments I got on my website were much better than the comments on either of those websites. But there seem to be some common misunderstandings underlying the bad comments, so I'll try to clear them up.

GENERALIZING RADIX SORT

The top comment on Hacker News essentially says "meh, this can't sort everything and we already knew that radix sort was faster." Firstly, I don't understand that negativity. My blog post was essentially "hey everyone, I am very excited to share that I have optimized and generalized radix sort" and your first response is "but you didn't generalize it all the way." Why the negativity? I take one step forward and you complain that I didn't take two steps? Why not be happy that I made radix sort apply to more domains than where it applied before?

So I want to talk about generalizing radix sort even more: The example of something that I don't handle is sorting a vector of std::sets. Say a vector of sets of ints. The reason why I can't sort th operator[]. std::sort does not have that problem because std::set provides compari

Comment Reblog Subscribe •••

There are two possible solutions here:

1. Where I currently use operator[], I could use std::next instead. So instead of writing container[index] I could write *std::next(container.begin(), index).
2. I could not use an index to indicate the current position, and only use iterators instead. For that I would have to allocate a second buffer to store one iterator per collection to sort.

Both of these approaches have problems. The first one is obviously slow because I have to iterate over the collection from the beginning every time that I want to look up the current element that I want to sort by. Meaning if I need to look at the first n elements to tell two lists apart, I need to walk over those elements n times, resulting in O(n^2) pointer dereferences. The normal set comparison operators don't have that problem because when they compare two sets, they can iterate over both in parallel. So when they need to look at n elements to tell two lists apart, they can do that in O(n).

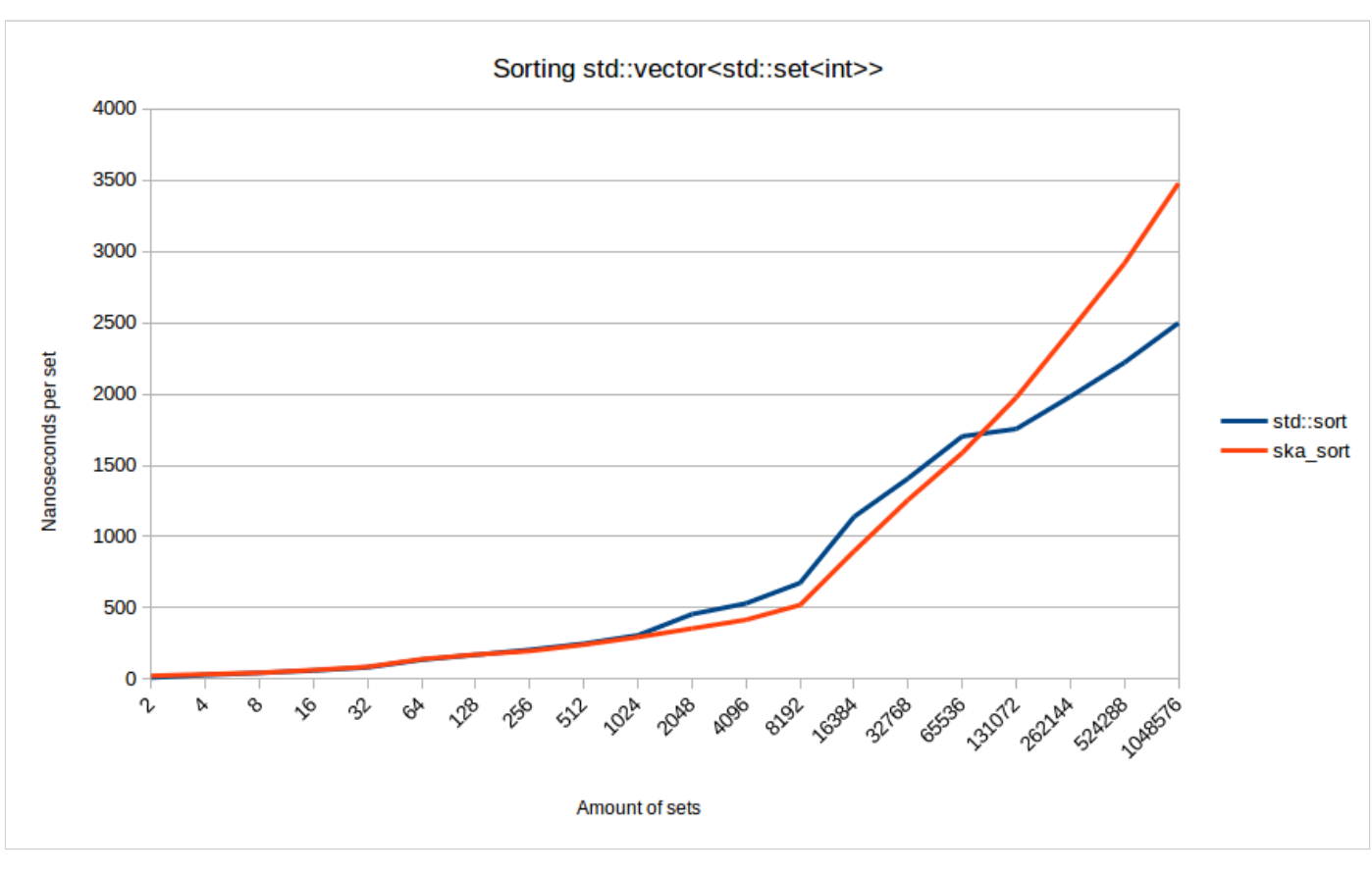
I also didn't want to allocate the extra memory that would be required for the second approach because I didn't want ska_sort to sometimes require heap allocations, and sometimes not require heap allocations depending on what type it is sorting.

The point is: I could easily generalize radix sort even more so that it can handle this case as well, but it doesn't seem interesting. Both approaches here have clear problems. I think you should just use std::sort here. So I'll limit ska_sort to things that can be accessed in constant time.

The other question is why you would want me to handle this. I stopped generalizing when I thought that I could handle all real use cases. Radix sort can be generalized more so that it can sort everything if you want it to. If you really need to sort a vector of std::sets or a vector of std::lists, then I can probably implement the second solution for you. But **the real question isn't whether ska_sort can sort everything, but whether it can sort your use case. And the answer is almost certainly yes.** If you really have a use case that ska_sort can not sort, then I can understand the criticism. But what do you want to sort that can not be reached in constant time?

That being said one thing that still needs to be done is that I need to allow customization of sorting behavior. Which is also what I wrote in my last blog post. Especially when sorting strings there are good use cases for wanting custom sorting behavior. Like case insensitive sorting. Or number aware sorting so that "foo100" comes after "foo99". I'll present an idea for that further down in this blog post. But the work there is not to generalize ska_sort further so that it can sort more data, but instead to give more customization options for the data that it can sort.

Before finishing this section, I actually quickly implemented solution 1 from the approaches for sorting sets above, and the graph is interesting:



ska_sort actually beats std::sort for quite a while there. std::sort is only faster when there are a lot of sets. If I construct the sets such that there is very little overlap between them, ska_sort is actually always faster. Does that mean that I should provide this code? I decided against it for now because it's not a clear win. I think if I did handle this, I would want to use the allocating solution because I expect a bigger win from that one.

APPLES AND ORANGES. COMPARING RADIX SORT AND STD::SORT

One criticism that I didn't understand at first was that I am comparing apples to oranges when I'm comparing my ska_sort to std::sort. You can see that same criticism voiced in that top Hacker New comment mentioned above. To me they are both sorting algorithms and who cares how they work internally? If you want to sort things, the only thing you care about is speed, not what the algorithm does internally.

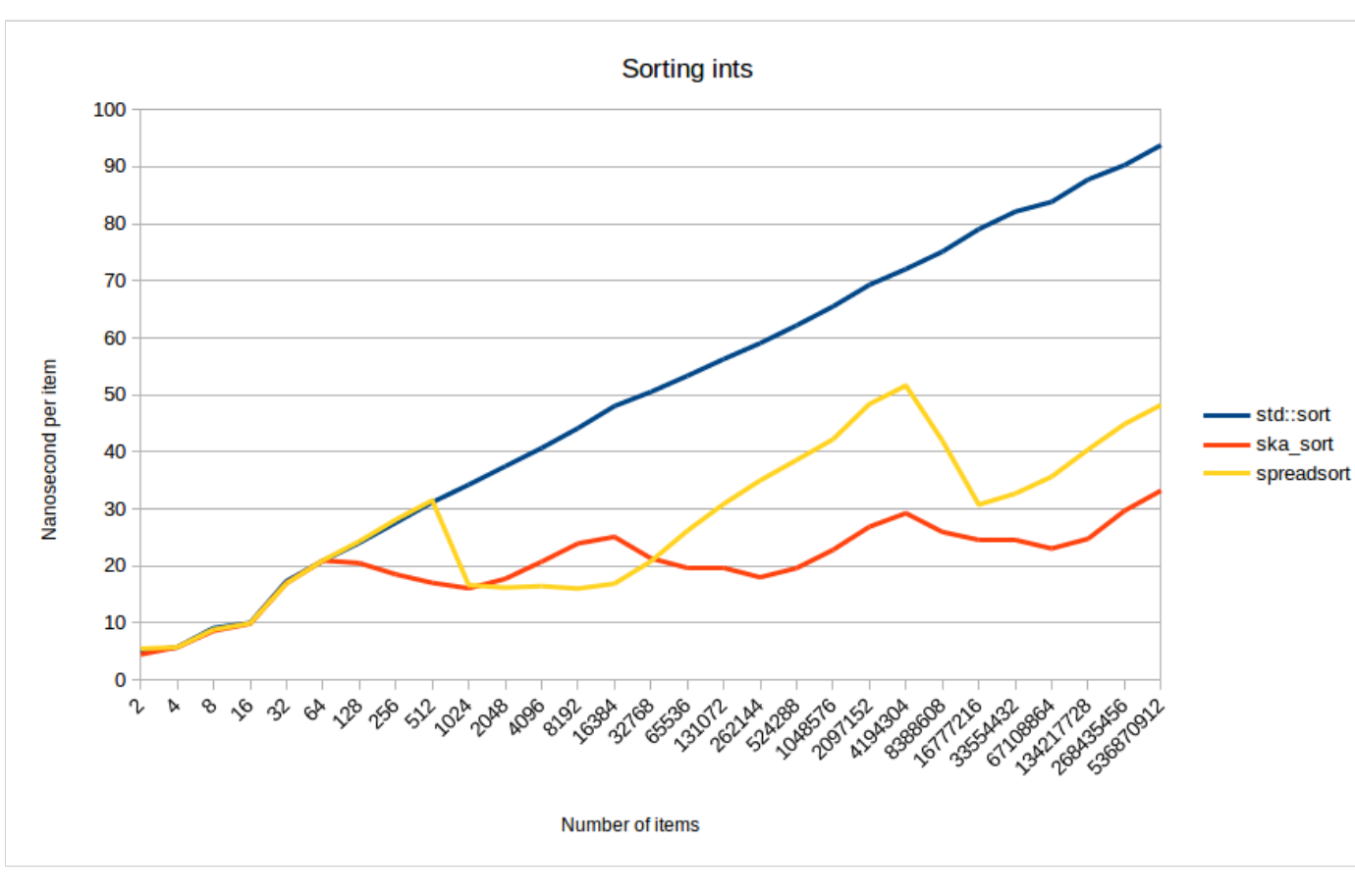
A friend of mine had a good analogy though: **Comparing a better radix sort to std::sort is like writing a faster hash table and saying "look at how much faster this is than std::map."**

However I contest that **what I did is not equivalent to writing a better hash table, but it is equivalent to writing the first general purpose hash table**. Imagine a parallel world where people have used hash tables for a while, but only ever for special purposes. Say everybody knows that you can only use hash tables if your key is an integer, otherwise you have to use a search tree. And then somebody comes along with the realization that you can store anything in a hash table as long as you provide a custom hash function. In that case it doesn't make sense to compare this new hash table to older hash tables, because older hash tables simply can't run most of your benchmarks because they only support integer keys.

Similarly the only thing that I could compare my radix sort to was std::sort, because older radix sort implementations couldn't run my benchmarks because they could literally only sort ints, floats and strings.

However the above argument doesn't make sense for me because I made two claims: I claimed that I generalized radix sort, and also that I optimized radix sort. For the second claim I should have provided benchmarks against other radix sort implementations. And also even though something like boost::spreadsort can't run all my benchmarks, I should have still compared it in the benchmarks that it can run. Meaning for sorting of ints, floats and strings. So yeah, I don't know what I was thinking there… Sometimes your brain just skips to the wrong conclusions and you never think about it a second time…

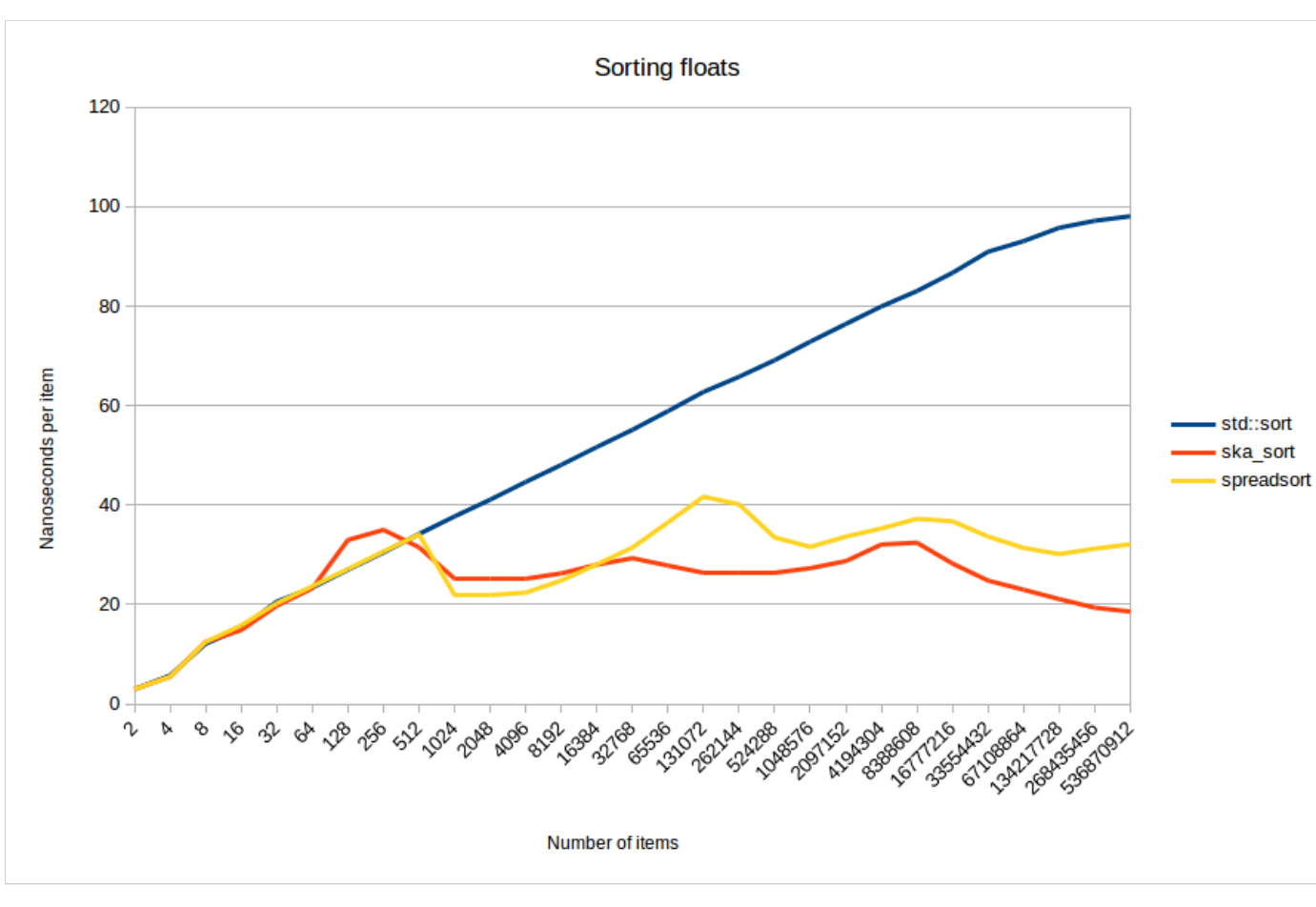
So anyways, here is ska_sort compared to boost::spreadsort when sorting uniformly distributed integers:



What we see here is that ska_sort is generally faster than boost::spreadsort. Except for that area between 2048 elements and 16384 elements. The reason for this is mainly that spreadsort picks a different number of partitions than I do. In each recursive step I split the input into 256 partitions. spreadsort uses more. It doesn't use a fixed amount like I do, so I can't tell you a simple number, except that it usually picks more than 256.

I had played around with using a different number of partitions in my first blog post about radix sort, but I didn't have good results that time. That time I found that if I used 2048 partitions, sorting would be faster if the collection had between 1024 and 4096 elements. In other cases using 256 partitions was faster. It's probably worth trying a variable amount of partitions like spreadsort uses. Maybe I can come up with an algorithm that's always faster than spreadsort. So there you go, a real benefit just from comparing against another radix sort implementation.

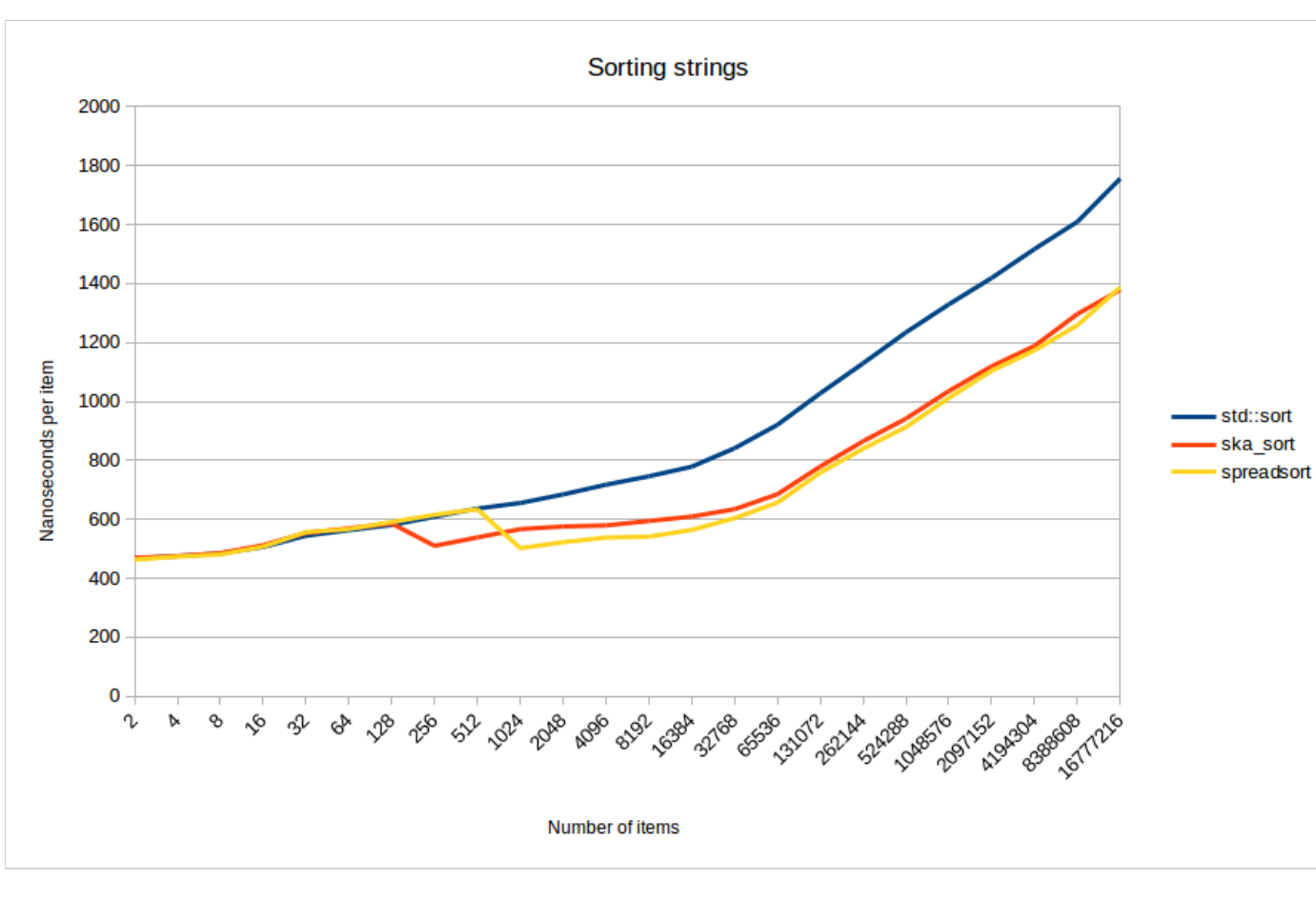
Let's also look at the graph for sorting uniformly distributed floats:



This graph is interesting for two reasons: 1. spreadsort has much smaller waves than when it sorts ints. 2. All of the algorithms seem to suddenly speed up when there are a lot of elements.

I have no good explanation for the second thing. But it is reproducible so I could do more investigation. My best guess is that this is because uniform_real_distribution just can't produce this many unique values. (I'm just asking for floats in the range from 0 to 1) So I'm getting more duplicates back there. I tried switching to a exponential_distribution, but the graph looked similar.

The reason for why spreadsort has smaller waves seems to be that spreadsort adjusts its algorithm based on how big the range of inputs is. When sorting ints it could get ints over the entire 32 bit range. When sorting floats it only gets values form zero to one. I need to look more into what spreadsort actually does with that information, but it does compute it and use it to determine how many partitions to use. But there's no time for looking into that. Instead let's look at sorting of strings:



This is the "sorting long strings" graph from my last blog post. And oops, that's embarrassing: spreadsort seems to be better at sorting strings than ska_sort. Which is surprising because I copied parts of my algorithm from spreadsort, so it should work similarly. Stepping through it, there seem to be two main reasons:

1. When sorting strings, I have to subdivide the input into 257 partitions: One partition for all the strings that are shorter than my current index, and 256 partitions for all the possible character values. I do that in two passes over the input: First split off all the shorter ones, second run my normal sorting algorithm which splits the remaining data into 256 partitions. spreadsort does this in one pass. There is no reason why I couldn't do the same thing. Except that it would complicate my algorithm even more because I'd need two slightly different versions of my inner loop. I'll try to do it when I get to it.
2. Spreadsort takes advantage of the fact that strings are always stored in a single chunk of memory. When it tries to find the longest common prefix between all the strings, it uses memcmp which will internally compare several bytes at a time. In my algorithm I have no special treatment for strings: It's the same algorithm for strings, deques, vectors, arrays or anything else with operator[]. This means I have to compare one element at a time because if you pass in a std::deque, memcmp wouldn't work. I could solve that by specializing for containers that have a .data() function. I would run into a second problem though: You might be sorting a vector of custom types, in which case memcmp would once again be the wrong thing. It still seems solvable: I just need even more template special cases for when the thing to be sorted is a character pointer in a container that has a data() member function. Doable, but adds more complexity.

So in conclusion spreadsort will stay faster than ska_sort at sorting strings for now. The reason for that is simply that I don't want to spend the time to implement the same optimizations at the moment.

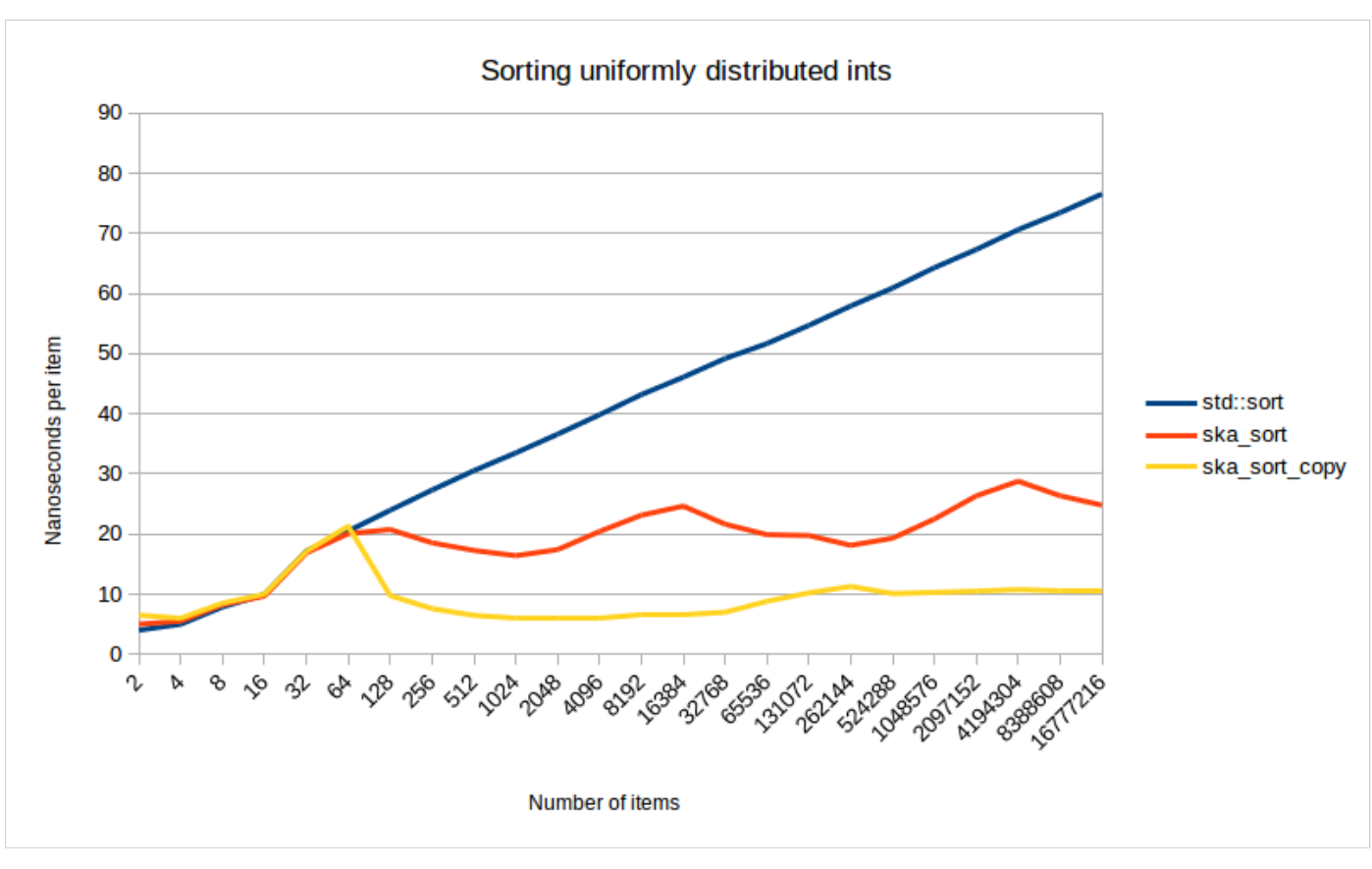
BIG O COMPLEXITY

The top reddit comment talked about something I wrote about the recursion count: It quotes from a part where I make two statements about the recursion count: 1. If I sort a million ints or a thousand ints, I always have to recurse at most four times. 2. If I can tell all the values apart in the first byte, I can stop recursing right there. The comment points out that these two statements apply to different ranges of inputs. Which, yes, they do. The comment makes fun of me for not stating that these apply to different ranges, then it contains some bike shedding about internal variable names and some wrong advice about merging loops that ping-pong between the buffers in ska_sort_copy. (when ping-ponging between two buffers A and B, you can't start the loop that reads B until the loop that reads A is finished writing to buffer B. Otherwise you read uninitialized data) I really don't understand why this is the top comment…

But I'll use this as an excuse to talk in detail about the recursion count and about big O complexity because that was a common topic in the comments. (including in the responses to that comment)

The point where I have to recurse into the second byte is actually more complex than you might think: I fall back to std::sort if a partition has fewer than 128 elements in it. That means that if the inputs are uniformly distributed on the first byte, I can handle up to 127*256 = 32512 values without any recursive calls. The 256 comes from the number of possible values for the first byte, and the 127 comes from the fact that if I create 256 partitions of 127 elements each, I will fall back to std::sort within each of those partitions instead of recursing to a second call of ska_sort.

Now in reality things are not that nicely distributed. Let me insert the graph again about sorting uniformly distributed ints:



The "waves" that you can see on ska_sort happen every time that I have to do one more recursive call. So what we see here is that in that middle wave, from 4096 to 16384 items, is when the pass that looks at the first byte creates more and more partitions that are large enough to require a recursive call. For example let's say that at 2048 elements I randomly get 80 items with the value 62 in the first byte. Then at 4096 bytes I randomly get 130 elements with the value 62 in the first byte. At 2048 elements I call std::sort directly, at 4096 elements I will do one more recursive call, splitting those 130 elements into another 256 partitions and then call std::sort on each of those.

Then after 16384 what happens is that those partitions are big enough that I can do nice large loops through them, and the algorithm speeds up again. That is until I have to recurse a second time starting at 512k items and I slow down again.

For integers there is a natural limit to these waves: There can be at most four of these waves because there are only four bytes in an int.

That brings us to the discussion about big O complexity. A funny thing to observe in the comments was that the more confident somebody was in claiming that I got my big O complexity wrong, the more likely they were to not understand big O complexity. But I will admit that the big O complexity for radix sort is confusing because it depends on the type of the data that you're sorting.

To start with I claim that a single pass over the data for me is O(n). This is not obvious from the source code because there are five loops in there, three of which are nested. But after looking at it a bit you find that those loops depends on two things: 1. The number of partitions, 2. the number of elements in the collection. So a first guess for the complexity would be O(n+p) where p is the number of partitions. That number is fixed in my algorithm to 256, so we end up with O(n+256) which is just O(n). But that 256 is the reason why ska_sort slows down when it has lots of small partitions.

Now every time that I can't separate the elements into partitions of fewer than 128 elements, I have to do a recursive call. So what is the impact of that on the complexity? A simple way of looking at that is to say it's O(n*b) where b is the number of bytes I need to look at until I can tell all the elements apart. When sorting integers, in the worst case this would be 4, so we end up with O(n*4) which is just O(n). When sorting something with a variable number of bytes, like strings, that b number could be arbitrarily big though. One trick I do to reduce the risk of hitting a really bad case there is that I skip over common prefixes. Still it's easy to create inputs where b is equal to n. So the algorithm is O(n^2) for sorting strings. But I actually detect that case and fall back to std::sort for the entire range then. So ska_sort is actually O(n log n) for sorting strings.

I like the O(n*b) number better though because the graph doesn't look like a O(n) graph. (ska_sort_copy however does look like a O(n) graph) The O(n*b) number gives a better understanding of what's going on. Then we can look at the waves in the graph and can say that at that point b increased by 1. And we can also see that b is not independent of n. (it will become independent of n once n is large enough. Say I'm sorting a trillion ints. But for small numbers b increases together with n)

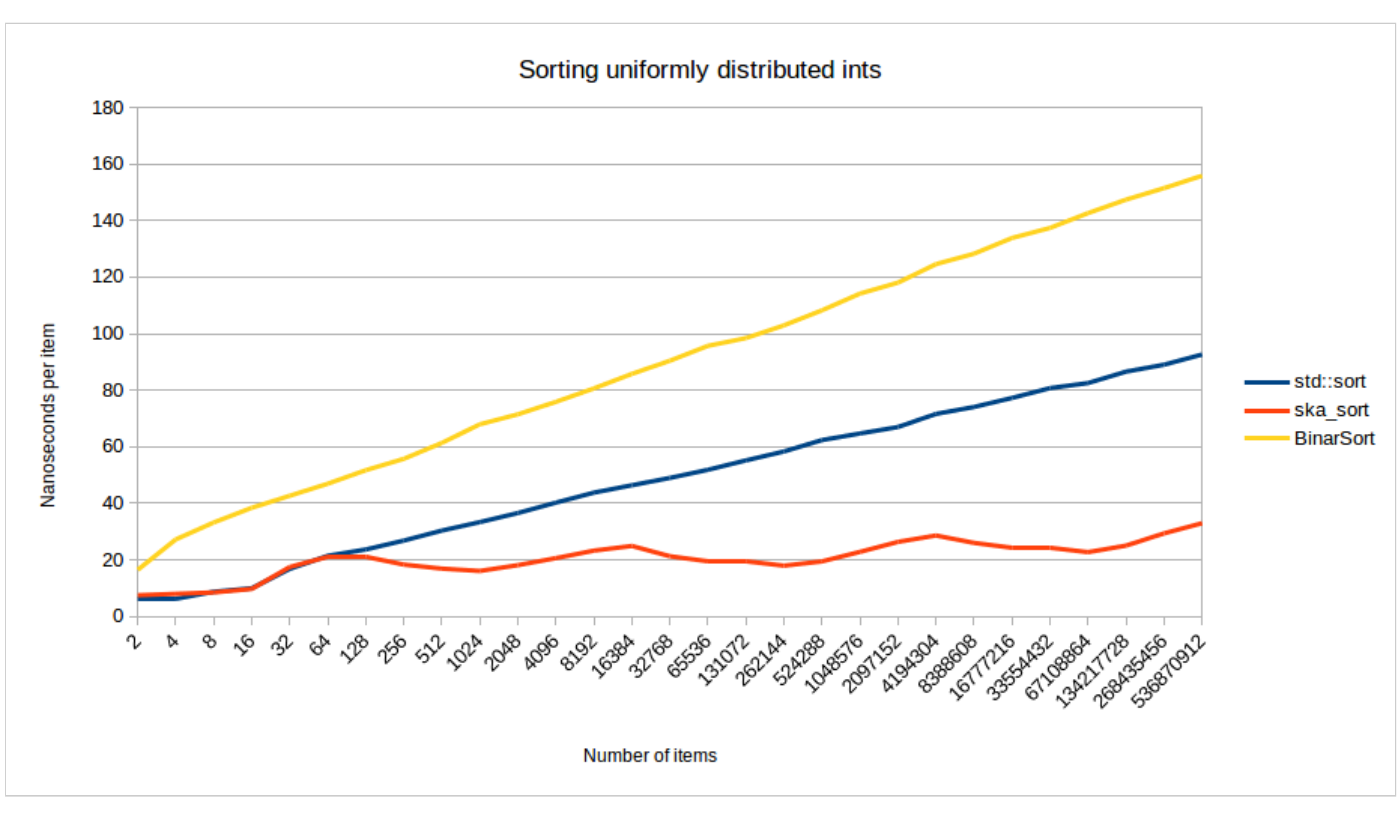
From this analysis you would think that my algorithm is slowest when all numbers are very close to each other. Say they're all close to 0. Because then I would have to look at all four bytes until I can tell all the numbers apart. In fact the opposite happens: My algorithm is fastest in these cases. The reason is that the first three passes over the data are very fast in this case because all elements have the same value for the first three bytes. Only the last pass actually has to do anything. (this is the "sorting geometric_distribution ints" graph from my last blog post where ska_sort ends up more than five times faster than std::sort)

Finally when looking at the complexity we have to consider the std::sort fallback. I will only ever call std::sort on partitions of fewer than 128 items. That means that the complexity of the std::sort fallback is not O(n log n) but it's O(n log 127), which is just O(n). It's O(n log 127) because a) I call std::sort on every element, so it has to be at least O(n), b) each of those calls to std::sort only sees at most 127 elements, so the recursive calls in quick sort are limited, and those recursive calls are responsible for the log n part of the complexity. If this sounds weird, it's the same reason that Introsort (which is used in std::sort) is O(n log n) even though it calls insertion sort (an O(n^2) algorithm) on every single element.

OTHER GENERALIZED RADIX SORTS

Some of the best comments I got were about other good sorting algorithms. And it turns out that other people have generalized radix sort before me.

One of those is BinarSort which was written by William Gilreath who commented on my blog post. BinarSort basically says "everything is made out of bits, so if we sort bits, we can sort everything." Which is a similar line of thinking that lead to me generalizing radix sort. The big downside with looking at everything as bits is that it leads to a slow sorting algorithm: For an int with 32 bits you have to do up to 31 recursive calls. Running BinarSort through my benchmark for sorting ints looks like this:



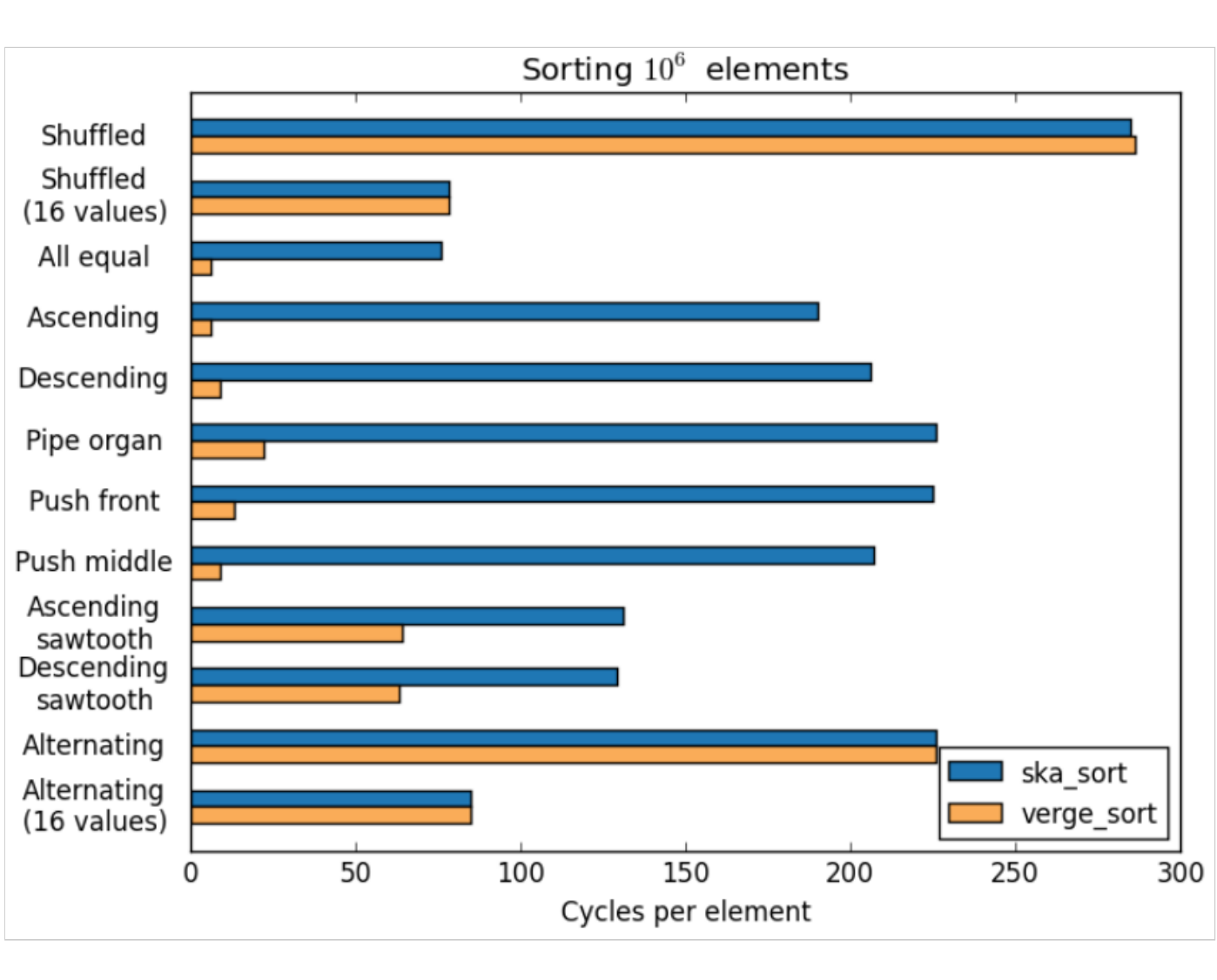
The first thing to notice is that BinarSort looks an awful lot as if it's O(n log n). The reason for that is the same reason that ska_sort doesn't look like a true O(n) graph: The number of recursive calls is related to the number of elements. BinarSort has to do up to 31 recursive calls. At the point where it reaches that number of recursive calls, you would expect the graph to flatten out. The quick sort which is used in std::sort would continue to grow even then. However it looks like you need to sort a huge number of items to get to that point in the graph. Instead you see an algorithm that keeps on getting slower and slower as it has to do more and more recursive calls, never reaching the point where it would turn linear.

The other big problem with BinarSort is that even though it claims to be general, it only provides source code for sorting ints. It doesn't provide a method for sorting other data. For example it's easy to see that you can't sort floats with it directly, because if you sort floats one bit at a time, positive floats come before negative floats. I now know how to sort floats using BinarSort because I did that work for ska_sort, but if I had read the paper a while ago, I wouldn't have believed the claim that you can sort everything with it. If you only provide source code for sorting ints, I will believe that you can only sort ints.

A much more promising approach is this paper by Fritz Henglein. I didn't read all of the paper but it looks like he did something very similar to what I did, except he did it five years ago. According to his graphs, his sorting algorithm is also much faster than older sorting algorithms. So I think he did great work, but for some reason nobody has heard about it. The lesson that I would take from that is that if you're doing work to improve performance of algorithms, don't do it in Haskell. The problem is that sorting is slow in Haskell to begin with. So he is comparing his algorithm against slow sorting algorithms and it's easy to beat slow sorting algorithms. I think that his algorithm would be faster than std::sort if he wrote it in C++, but it's hard to tell.

VERGESORT

A great thing that happened in the comments was that Morwenn adapted his sorting algorithm Vergesort to run on top of ska_sort. The result is an algorithm that performs very well on pre-sorted data while still being fast in random data.



This is the graph that he posted. ska_sort is just ska_sort by itself, verge_sort is a combination of verge sort and ska_sort. Best of all, he posted a comment explaining how he did it.

So that is absolutely fantastic. I'll definitely attempt to bring his changes into the main algorithm. There might even be a way to merge his loop over the data with my first loop over the data, so that I don't even have to do an extra pass.

This brings me to future work:

FUTURE WORK

Custom sorting behavior is my next task. I don't have a full solution yet, but I have something that can handle case-insensitive sorting of ASCII characters and it can do number-aware sorting. The hope is that something like Unicode sorting could be done with the same approach. The idea is that I expose a customization point where you can change how I use iterators in collections. You can change the return value from the iterator, and you can change how far the iterator will advance. So for case insensitive sorting you could simply return 'a' from the iterator when the actual value was 'A'.

The tricky part is number aware sorting. My current idea is that you could return an int instead of a char, and then advance the iterator several positions. You would have to be a bit tricky with the int that you return because you would want to return either a character or an int. I could add support for std::variant (should probably do that anyway) but we can also just say that for characters, we just cast it to an int, and for numbers we return the lowest int plus the number. So for "foo100" you would return the integers 'f', 'o', 'o', INT_MIN+100. And for "foo99" you would return the integers 'f', 'o', 'o', INT_MIN+99. Then you would advance the iterator one position for the first three characters, and three positions for the number 100 and two positions for the number 99. One tricky part on this is that you have to always move the iterator forward by the same distance when elements have the same value. Meaning if two different strings have the value INT_MIN+100 for the current index, they both have to advance their iterators by three elements. Can't have one of them advancing it by four elements. I need that assumption so that for recursive calls, I only need to advance a single index. So I won't actually store the iterators that you return, but only a single index that I can use for all values that fell into the same partition. I think it's a promising idea. It might also work for sorting Unicode, but that is such a complicated topic that I have no idea if this will work or not. I think the only way to find out is to start working on this and see if I run into problems.

The other task that I want to do is to merge my algorithm with verge sort so that I can also be fast for pre-sorted ranges.

The big problem that I have right now is that I actually want to take a break from this. I don't want to work on this sorting algorithm for a while. I was actually already kinda burned out on this before I even wrote that first blog post. At that point I had spent a month of my free time on this and I was very happy to finally be done with this when I hit "Publish" on that blog post. So sorry, but the current state is what it's going to stay at. I'm doing this in my spare time, and right now I've got other things that I would like to do with my spare time. (Dark Souls III is pretty great, guys)

That being said I do intend to use this algorithm at work, and I do expect some small improvements to come out of that. These things always get improved as soon as you actually start using them. Also I'll probably get back to this at some point this year.

Until then you should give ska_sort a try. This can literally make your sorting two times faster. Also if you have data that this can't sort, I am very curious to hear about that. Here's a link to the source code and you can find instructions on how to use it in my last blog post.

Share this:

Twitter Facebook

Related

I Wrote a Faster Sorting Algorithm	Investigating Radix Sort	Beautiful Branchless Binary Search
December 27, 2016	December 2, 2016	April 27, 2023
In "Programming"	In "Programming"	In "Programming"

PUBLISHED: January 17, 2017
FILED UNDER: Programming
TAGS: C++ : ska_sort

2 Comments to "Faster Sorting Algorithm Part 2"

lyl *March 7, 2018 at 21:48*

Nice article. Thanks for sharing this. I have a question about the performance number though. In your graph of sorting uniformly distributed ints, ska_sort takes about 20 ns to sort an element when the total number of elements is less than 1M. However, in the VergeSort graph, ska_sort takes almost 300 cycles (about 100-150 ns?) to sort an element in the "shuffled" distribution. Do you know why there is such a large gap between the measurements? Thanks!

REPLY

Malte Skarupke *March 12, 2018 at 19:14*

That's a good question, and I have no idea. I didn't look at the different scales because the two measurements were done by different people using different benchmarks on different computers.

That being said I just downloaded Morwenn's code and ran this benchmark: https://github.com/Morwenn/cpp-sort/blob/master/benchmarks/bench.cpp

Which for me locally (running on the same machine as the other benchmark above) says that ska_sort runs in 37 cycles per item. Which would be even faster than my own benchmark. Because 20ns is equal to 70 cycles on my 3.5 gigahertz computer.

So after looking into it, I still don't have a good answer for you. It could be 37 cycles, could be 70 cycles, or it could be 300 cycles. But what I can tell you is that when running Morwenn's benchmark for random data, ska_sort easily beats all the other sorting algorithms. So it's not like my benchmarks were hand-tuned for my algorithm.

REPLY

Leave a comment

This site uses Akismet to reduce spam. Learn how your comment data is processed.

« Previous Post Next Post »

Search

RECENT POSTS

How I use LLMs to program
Transform Matrices are Great and You Should Understand Them
Two Kids Put Me on a Two Sleep Schedule
Beautiful Branchless Binary Search
Fine-grained Locking with Two-Bit Mutexes

ARCHIVES

April 2024
October 2023
September 2023
April 2023
December 2022
September 2022
June 2022
February 2022
January 2022
October 2021
July 2021
April 2021
January 2021
November 2020
October 2020
August 2020
July 2020
June 2020
May 2020
April 2020
March 2020
January 2020
December 2019
September 2019
August 2019
June 2019
April 2019
March 2019
June 2018
May 2018
April 2018
January 2018
December 2017
November 2017
October 2017
September 2017
August 2017
February 2017
January 2017
December 2016
November 2016
June 2016
April 2016
March 2016
February 2016
December 2015
September 2015
July 2015
June 2015
May 2015
February 2015
January 2015
December 2014
November 2014
October 2014
September 2014
August 2014
June 2014
May 2014
April 2014
March 2014
February 2014
January 2014
October 2013
September 2013
August 2013
May 2013
February 2013
January 2013
December 2012
November 2012
October 2012
August 2012
July 2012
April 2012
March 2012
February 2012
January 2012
October 2011
September 2011
August 2011
July 2011
June 2011
May 2011

CATEGORIES

Children
Games
Links
Math
Politics and Economics
Programming
Uncategorized

META

Register
Log in
Entries feed
Comments feed
WordPress.com

Search